

Editorial 1337

Aureliu Valentin Antonie, Adrian Andrei Roşu

December 2025

(Subtask 1) Pentru acest subtask, putem parcurge toate subsecvenţele din vector. Pentru fiecare subsecvenţă, verificăm ca restul împărţirii sumei acesteia la k să fie egal cu p . Dacă această condiţie este verificată şi maximul subsecvenţei este mai mare decât maximul global, actualizăm maximul global. Dacă mai multe subsecvenţe conţin maximul global, alegem prima subsecvenţă de lungime minimă. Prin modul de generare al indicilor care delimitează subsecvenţa ($i = 1..n$, $j = i..n$), subsecvenţa de lungime minimă aleasă va fi şi subsecvenţa minimă cu indicele j (şi i) cel mai mic. Dacă condiţia din enunţ nu este verificată de nicio subsecvenţă, afişăm **Nu exista**.

Complexitate $O(n^2)$ - 20 de puncte

(Subtask-uri 2 şi 3)

Cerinţa 1. Printr-o parcurgere ($j = 1..n$), fixăm indicele j . Generăm sumele parţiale $sum_j = \left(\sum_{i=1}^j v_i\right) \bmod k$. Observăm că o subsecvenţă $v_i, v_{i+1}, \dots, v_j$ cu suma mod k egală cu p respectă condiţia:

$$(sum_j - sum_{i-1} + k) \bmod k = p, \quad i < j$$

Având j fixat (şi cunoscând sum_j), vrem să găsim toţi indicii i cu proprietatea anterioară (acest lucru va fi util mai ales pentru a doua cerinţă). Pentru a face acest lucru, vom ţine un hashmap, cu cheia rest şi valoarea reprezentată de mulţimea ordonată a tuturor indicilor j , cu proprietatea $sum_j = rest$. Pentru prima cerinţă (afiarea elementului special X), ne interesează cea mai lungă subsecvenţă cu suma mod k egală cu p , care este delimitată superior de j . Pentru indicele j fixat, această subsecvenţă este delimitată inferior de primul indice i , cu proprietatea:

$$sum_{i-1} = (sum_j - p + k) \bmod k, \quad i < j$$

(putem afla acest indice uşor din hashmap). Pentru a afla maximul subsecvenţei putem folosi o structură de date de tip Range Minimum Query, arbori de intervale sau şmenul lui Mars etc. Dacă condiţia din enunţ nu este verificată de nicio subsecvenţă, afişăm **Nu exista**.

Complexitate $O(n \log n)$ sau $O(n)$ – ambele complexităţi duc la punctaj maximal.

Cerinţa 2. Printr-o nouă parcurgere ($j = 1..n$), fixăm indicele j . Vrem să găsim subsecvenţa de lungime minimă care este delimitată de j şi conţine

elementul X . Pentru acest lucru, căutăm binar în mulțimea ordonată a indicilor i , cu proprietatea de mai jos (folosim hashmap-ul construit anterior).

$$sum_{i-1} = (sum_j - p + k) \bmod k, \quad i < j$$

Pentru a verifica dacă subsecvența $v_i, v_{i+1}, \dots, v_j$ conține elementul X , ne folosim de Range Minimum Query sau arbori de intervale.

Complexitate Range Minimum Query $O(n \log n)$ – 100 de puncte

Complexitate arbori de intervale $O(n \log^2 n)$ – 50 de puncte

O alta implementare pentru a doua cerință constă în recunoașterea structurii unei secvențe valide. O secvență validă pentru un indice j fixat trebuie să înceapă de la un indice i astfel încât ea să conțină elementul maxim și, desigur, să respecte proprietatea din enunț.

$$(i, v[pos], j), \text{ cu } (sum_j - sum_{i-1} + k) \bmod k = p \text{ și } i \leq pos \leq j$$

Putem astfel pentru un indice j fixat să cautăm binar cea mai mare poziție $pos \leq j$ pentru care $v[pos] = \max$. Pornind de la această nouă poziție putem căuta binar încă o dată, acum în hashmap-ul construit anterior cea mai mare poziție i pentru care $i \leq pos$ și $(sum_j - sum_{i-1} + k) \bmod k = p$. Printre intervalele de forma (i, j) construite astfel este garantat să găsim răspunsul la problemă.

Complexitatea finală este $O(n \log n)$ și obține 100 de puncte fără a necesita cunoașterea unei structuri de date mai avansată pentru maxime în timp logaritm sau constant.